Document 12 — Production AI Engineering

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

11_multi_agent_systems (Project 4 done)

How to Read & Practice This Document

- **What:** turning a script that works into a real service other people can run, trust, and pay for.
- **Why:** "it works on my computer" isn't a finished product — this document is the whole gap between a portfolio demo and a job-ready system.
- **When:** the moment anyone besides you needs to use what you built, or it needs to run without you watching it.
- **How to practice:** 1. Go through FastAPI's tutorial hands-on, in your own editor — this tool rewards typing along, not just reading. 2. Do the **Basic/Intermediate** endpoint exercises closed-book where you can. 3. Do **Real-world/Edge case** exercises against a real SQLite file, not a fake one. 4. Do the **Failure/Production** exercises for real: actually kill the database connection, actually build and run the Dockerfile. Don't skip to "I get the idea." 5. Use **Hint 1 → Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, run `docker history` on your own image, and check no secret is baked in. If you haven't checked, you're not done.

Jump to: Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Project 4 (Doc11) works — but right now it only works when you personally type a command in your own terminal, on your own computer. Nobody else can use it. It can't run unattended. It has no memory of what happened five minutes ago unless your terminal is still open. This document is the gap between "it works on my machine" and "other people can actually use this, trust it, and it keeps working while nobody's watching."

Closing that gap is really four separate, boring-sounding problems, and this document takes them one at a time. First, something else needs to be able to *call* your agent — not by running a Python script, but over the network, the way every real app talks to every other real app. That's **FastAPI**: it

turns your agent into an HTTP endpoint with a typed, checked request and reply, the same Pydantic idea from Doc04/06, just applied one layer up at the web boundary. Second, once requests come from strangers instead of just you, you need a permanent, look-up-able record of what happened — which is what a **database** gives you that a Python list in memory never could: it survives a restart, and you can ask it "what happened at 3 AM?" after the fact. Third, your code needs to run the same way on a server that's never seen it before as it does on your laptop — that's what **Docker** solves, by packaging your app and everything it depends on into one portable, repeatable unit. And fourth, once real traffic starts hitting a real service, cost starts to matter in a way it never did in a terminal — which is where smart, careful **caching** comes in, saving real money without ever risking a wrong answer.

That's the whole story: **FastAPI** is the front door. **The database** is the memory. **Docker** is what makes it run anywhere, not just your machine. **Caching** is what keeps it affordable once real traffic shows up. And the **Build Task** below takes Project 4 — your multi-agent system — and wraps it in exactly these four things, so it stops being a script you run and becomes "Project 4 as an API": a real service, reachable over HTTP, backed by a database, and shippable in a container.

Topic tiers (full detail here, referenced from CURRICULUM.md)

- **Must-know:** FastAPI endpoints, request/response models, error handling, keeping secrets safe, structured logging, retries/timeouts, basic rate limiting.
- **Important:** database schema for conversation/run history, packaging with Docker, caching replies/embeddings.
- **Later (beyond this document):** login/permissions, basic CI/CD, queues for long-running agent work, scaling across servers.
- **Optional:** extra cost-saving (prompt caching, picking cheaper models for easier tasks), running in multiple regions.

Core Concepts (read this first — everything you need is here)

FastAPI: request/response shapes are Pydantic again, one layer up

FastAPI is a Python library for building web APIs. An API route is a Python function that runs when some other program sends an HTTP request to a URL, like `POST /run-task`. FastAPI routes take in typed request shapes and give back typed response shapes. This is the exact same Pydantic idea from Doc04 and Doc06, but now used at the HTTP boundary (the edge where outside requests enter your code) instead of for tool arguments. A route function declares a Pydantic model as its input. FastAPI checks the incoming JSON against that model *before* your function's code even runs. If the JSON does not match, FastAPI automatically sends back a clear **422** error ("Unprocessable Entity" — the request was understood, but the data is wrong). You don't write that checking code yourself.

Why this matters: without a checked shape, every route has to test by hand "is `task` there? is it a string? is it empty?". People forget some of these checks, and bad data then reaches your agent or your database. With Pydantic models, the API's contract (what it accepts and what it returns) is written in one place. FastAPI even builds interactive docs from your type hints — open `http://localhost:8000/docs` and you can try every route in the browser. **How error handling connects to Doc02:** a route should turn internal failures into the right HTTP status code. A bad-input problem is a **4xx** (the client's fault — sending the same request again will fail again). An unexpected internal error is a **5xx** (the server's fault — a retry later may work). A 5xx reply should never show a raw error trace to the client, because a trace can show file paths, SQL, or even secrets. Log the details on the server, and send a plain message to the client.

When to use it: any time another program — a web page, a mobile app, another service, a teammate's script, or another agent — needs to call your code over the network. **When NOT to:** a script only you run by hand (a plain CLI is simpler), or a nightly batch job that nobody calls (a scheduled script is enough). Also, don't put a slow, 5-minute agent run behind a normal request that waits for the answer — clients and proxies time out. For that, the route should start the job and return a run ID, and the client checks back later (the "queues" item under **Later** in the Topic tiers above).

Which status code to return, by situation:

Situation	What to do	Why	Small example
Request JSON is missing a field or has the wrong type	Nothing — let FastAPI return **422**	Pydantic already checked it; your route never runs	`task: str` missing → 422 automatically
JSON shape is fine, but the value can't be used (e.g. unsupported language)	Raise **400** with a clear message	Client's fault; the message tells them what to fix	`raise HTTPException(400, "language must be 'en' or 'ur'")`
Client asks for something that doesn't exist	Raise **404**	Tells the client "wrong ID", not "server broken"	`GET /runs/abc` with no such row
Client sends too many requests too fast	Return **429**	Protects your API and your OpenAI bill	Build Task rate limit
A bug, a dead database, or an unexpected error in your code	Log the full error, return **500** with a plain message	Client can't fix it; the trace must stay on the server	`logger.exception(...)`, then `HTTPException(500, "Internal error")`
An outside service you depend on (OpenAI) is	Return **503** (or 502/504)	Tells the client "try again later";	`except openai.APITimeoutError:` → 503

Situation	What to do	Why	Small example
down or timed out		uses Doc02's retry rules	

How it works — one full route:

```
import logging
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

logger = logging.getLogger(__name__)
app = FastAPI()

class TaskRequest(BaseModel):
    task: str = Field(min_length=1, max_length=2000)
    # ge/le = "greater/less or equal"
    max_steps: int = Field(default=5, ge=1, le=20)

class TaskResponse(BaseModel):
    run_id: str
    result: str

class TaskInputError(Exception):
    """The request shape is fine, but the task can't be done."""

@app.post("/run-task", response_model=TaskResponse)
def run_task(req: TaskRequest) -> TaskResponse:
    try:
        # your Project 4 code
        run_id, result = run_pipeline(req.task, max_steps=req.max_steps)
    except TaskInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        # full trace goes to server logs only
        logger.exception("run_task failed")
        raise HTTPException(
            status_code=500, detail="Internal error. Please try again later."
        )
    return TaskResponse(run_id=run_id, result=result)
```

Run it with `uvicorn main:app --reload` (`uvicorn` is the web server program that actually listens on a port and passes requests to your `app`). The broad `except Exception` here is not a break of Doc01's "catch only what you can handle" rule. At the API boundary, you *can* do something useful with any error: log it fully and turn it into a safe 500. Inside your own helper functions, keep catching specific errors.

Real-world examples, by situation:

A support desk looks up one ticket — 404 when it doesn't exist:

```
@app.get("/tickets/{ticket_id}")
def get_ticket(ticket_id: int) -> dict:
    # returns dict | None (Doc01 type-hint pattern)
    ticket = find_ticket(ticket_id)
    if ticket is None:
        raise HTTPException(status_code=404, detail="Ticket not found")
    return ticket
```

FastAPI also checks the path: `GET /tickets/abc` returns 422, because `ticket_id` must be an `int`.

Hide internal fields from the client: your pipeline returns a dict with `result`, `cost_usd`, and the full `system_prompt`. With `response_model=TaskResponse`, FastAPI sends only the fields declared in `TaskResponse`. Extra fields are dropped, so the system prompt never leaks by accident.

A mobile app sends a date in the wrong format: `{"due": "tomorrow"}` against `due: date` gives a 422 whose body lists the exact field and problem. The app developer can fix their code from the error message alone, without asking you.

`def` or `async def`? If your route calls normal blocking code (the sync OpenAI client, `sqlite3`, `requests`), write a plain `def` route — FastAPI runs it in a thread pool, so other requests keep moving. Write `async def` only when everything inside uses `await` (see Doc08b).

Where you'll meet it: the Practice Exercises below, then Project 5, which puts Project 4 (the multi-agent system from Doc11) behind `POST /run-task`. Doc13 tests these routes with FastAPI's `TestClient`. In a multi-agent system, the API route is the one front door: the request model is the first input the first agent receives, so a clean, checked shape here protects every agent behind it. In bigger systems, agents can even run as separate services and call each other through routes like this. Real systems built this way: chatbot backends, support-desk APIs, RAG "ask our company docs" endpoints, and internal tools that other teams call.

A common mistake: writing `async def` for a route and then calling a blocking function inside it, for example the sync `client.chat.completions.create(...)` or `time.sleep(...)`. What it causes: the blocking call freezes FastAPI's single event loop, so while one request waits 10 seconds for OpenAI, *every other request* waits too — even `/health`. How to spot it: requests that should be fast become slow only when another slow request is running at the same time. Fix it by using a plain `def` route, or by using the async client (`AsyncOpenAI`) with `await`.

Quick cheat sheet:

- Request body in = a Pydantic model parameter; response out = `response_model=...`.
- Bad shape → FastAPI's automatic 422. Bad value → your 400. Missing thing → 404.

- Unexpected error → log the full trace on the server, return a plain 500. Never send a trace to the client.
- Blocking code inside → `def` route. All-`await` code inside → `async def` route.
- Open `/docs` to test your routes in the browser for free.

Database storage: what to save, and why start with SQLite

A database is a program that stores data in organized tables and lets you search it with a query language (SQL). It gives you something a Python list in memory can't: data that survives a restart, and that you can look up later (which run failed? what did agent X return at 3 AM?). **SQLite** is a real, fully working database that runs inside your Python process — there is no separate server program, and the whole database is one file (like `runs.db`). Python ships with the `sqlite3` module, so you don't install anything.

Why this matters: with a list in memory, a restart, a crash, or a new deploy wipes all history. Then a user says "my request at 3 AM gave a wrong answer", and you have nothing to look at. **Why start with SQLite specifically:** you can learn schema design (planning your tables and columns), queries, and transactions without the extra work of running a separate database server. Later you can move to Postgres or MySQL using the same SQL skills, once you actually need more than SQLite can handle. **What to save for an agent system specifically:** enough to rebuild what happened in a run, after the fact — input, output, the steps in between, status, timestamps. Do not save secrets. Do not save so much raw data (full prompts on every step, huge tool results) that storage cost becomes its own problem.

When to use what:

Situation	What to do	Why	Small example
Learning, a single-server app, a small internal tool	**SQLite**	One file, no server to run, fast enough for many small apps	`sqlite3.connect("runs.db")`
Several app copies (containers/servers) must write to the same data	Move to **Postgres**	A SQLite file lives on one disk; many writers across machines can't share it safely	`DATABASE_URL=postgresql://...`
Data is only needed during one request	**Don't** use a database — a local variable is fine	Saving throw-away data costs time and storage	A temporary list of search hits

Situation	What to do	Why	Small example
You must answer "what happened in run X?" later	Save a row per run, and a row per agent step	A run you can't look up can't be debugged	`runs` table + `run_steps` table
A value is a secret (API key, password)	**Never** store it in the run history	Anyone with read access to the DB would see it	Store `"model": "gpt-4.1-mini"`, not the key
Your SQL grows into many tables and joins	Consider **SQLAlchemy** (a Python library that maps tables to classes)	Less hand-written SQL; easy to switch SQLite → Postgres	See Go Deeper below

How it works — create a table, save a run, read it back:

```
import sqlite3
import uuid
from datetime import datetime, timezone

# check_same_thread=False: allow use from FastAPI's threads
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row   # rows behave like dicts
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id         TEXT PRIMARY KEY,
        input      TEXT NOT NULL,
        output     TEXT,
        status     TEXT NOT NULL,
        created_at TEXT NOT NULL
    )
""")

def save_run(task: str, output: str | None, status: str) -> str:
    run_id = str(uuid.uuid4())
    now = datetime.now(timezone.utc).isoformat()
    sql = (
        "INSERT INTO runs (id, input, output, status, created_at) "
        "VALUES (?, ?, ?, ?, ?)"
    )
    # a transaction: commits if the block succeeds, rolls back if it raises
    with conn:
        conn.execute(sql, (run_id, task, output, status, now))
    return run_id
```

```
def get_run(run_id: str) -> dict | None:
    sql = "SELECT * FROM runs WHERE id = ?"
    row = conn.execute(sql, (run_id,)).fetchone()
    if row is None:
        return None
    return dict(row)
```

Three details matter here. The `?` marks are **placeholders**: `sqlite3` puts the values in safely, so a task text like `It's done'; DROP TABLE runs; --` is stored as plain text, not run as SQL. `with conn:` wraps the write in a **transaction** (all of it saves, or none of it does), so a crash never leaves half a row. Timestamps are saved in UTC as ISO text, so sorting by `created_at` works. One shared connection is fine for learning and light traffic. Under real concurrent traffic, open a connection per request, or move to SQLAlchemy/Postgres.

Real-world examples, by situation:

A multi-agent run, one row per agent step (Doc11 / Project 5):

```
conn.execute("""CREATE TABLE IF NOT EXISTS run_steps (
    run_id TEXT NOT NULL REFERENCES runs(id),
    step_no INTEGER NOT NULL,
    agent TEXT NOT NULL,          -- "researcher", "writer", "reviewer"
    output TEXT,
    created_at TEXT NOT NULL
)""")
```

When the final article is wrong, you run `SELECT agent, output FROM run_steps WHERE run_id = ? ORDER BY step_no` and see exactly which agent went wrong.

A support desk saves structured data: the triage result is a Pydantic model. Save it as JSON text with `result.model_dump_json()`, and load it back with `TriageResult.model_validate_json(row["output"])`. The shape stays checked on the way in and on the way out.

A nightly report job: a scheduled script sums yesterday's cost: `SELECT COUNT(*), SUM(cost_usd) FROM runs WHERE created_at >= ?`. This question is impossible to answer if runs only lived in memory.

Agent memory across conversations: Project 10 uses LangGraph's `SqliteSaver`, which stores the graph's state in a SQLite file, so a conversation continues after the program restarts.

Where you'll meet it: the Real-world and Failure exercises below, then Step 2 of Project 5 (every run saved and findable by ID). Project 10 uses SQLite for long-term agent memory. Doc13 reads saved runs to find failures, and Doc19 compares cost and quality over time from the same kind of

table. In a multi-agent system, the database is the shared record: each agent writes its step under the same `run_id`, so one query shows the whole path a task took. Real systems: support-ticket history, chatbot conversation logs, RAG "which documents were used for this answer" records, and job status tables for long-running work.

A common mistake: building SQL with an f-string, like `conn.execute(f"SELECT * FROM runs WHERE id = '{run_id}'")`. What it causes: any value with a `'` in it breaks the query, and a user can send text that changes the query itself (this is called **SQL injection**) — reading or deleting data they should never touch. With LLM apps this is extra risky, because model output can also contain any text. How to spot it: search your code for `execute(f"` or `+` inside SQL strings. Every value must go through `?` placeholders instead.

Quick cheat sheet:

- Start with SQLite (one file, built into Python); move to Postgres when several servers must write.
- Always use `?` placeholders — never f-strings — to put values into SQL.
- Wrap writes in `with conn:` so a failure rolls back instead of saving half a row.
- Save input, output, status, timestamps (UTC), and one row per agent step — never secrets.
- Store Pydantic results as JSON text with `model_dump_json()`, and read them back with `model_validate_json()`.

Docker: what a container actually is, and why it exists

A container packages your app together with everything it needs to run (the Python version, installed packages, system-level dependencies) into one portable unit that runs the same way no matter what's installed on the computer running it. Two words to keep apart: an **image** is the packaged, read-only recipe result (built once with `docker build`), and a **container** is one running copy of that image (started with `docker run`). One image can start many containers.

Why this solves a real, common problem: "it works on my computer" almost always means the computer has some dependency, version, or setting your code silently depends on, that isn't written down anywhere. A container makes every dependency clear in a `Dockerfile`, so if it builds and runs from that file, it will run the same way on any computer with Docker installed — including a production server that's never seen your code before. Without it, every new server needs someone to install the right Python and packages by hand, and one small difference can break the app.

When to use it / when NOT to:

Situation	What to do	Why	Small example
Your API must run on a server, a cloud platform, or a teammate's laptop	Write a `Dockerfile` and ship the image	Same Python and packages everywhere	`docker build -t contentforge .`

Situation	What to do	Why	Small example
You are still writing and changing code every minute on your laptop	Use your normal venv (Doc01) and `uvicorn --reload`	Rebuilding an image on every edit is slow	`source .venv/bin/activate`
The app needs a secret (API key)	Pass it at **run** time with `-e` or `--env-file`	Anything in the image can be read by anyone who has the image	`docker run --env-file .env contentforge`
The app writes data that must survive (SQLite file)	Mount a **volume** (a folder kept outside the container)	A container's own files are lost when the container is removed	`-v ./data:/app/data`
You need API + database + cache running together	Use `docker compose`	One command starts and connects all of them	`docker compose up`
A tiny one-off script you run once	Skip Docker	The packaging costs more time than it saves	`python fix_rows.py`

How a `Dockerfile` works, step by step: each line (`FROM`, `COPY`, `RUN`, `CMD`) creates a saved, unchanging layer, stacked on top of the one before it. `FROM python:3.11-slim` starts from a small base image with Python already installed. `COPY requirements.txt .` then `RUN pip install -r requirements.txt` installs your packages *before* copying the rest of your app's code — this order is on purpose: Docker reuses a saved layer if nothing that built it changed, so if you edit your app's code but not `requirements.txt`, the (usually slow) install step gets skipped on the next build, instead of running every time. `COPY . .` copies your app's code. `CMD [...]` says what runs when the container starts.

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# build the image
docker build -t contentforge .
docker run -p 8000:8000 --env-file .env \
```

```
    -v "$(pwd)/data:/app/data" contentforge
# same reply as your laptop
curl localhost:8000/health
```

`-p 8000:8000` connects port 8000 on your computer to port 8000 inside the container. `--env-file .env` gives the container its secrets at run time. The `.env` file stays on your computer and is never copied into the image.

Why secrets never belong in the image: anything baked into a Docker image layer (by `COPY`-ing a `.env` file, or hardcoding a value in `ENV`) stays permanently inside that image — visible to anyone who can look at it (`docker history` shows what's in each layer), even if a *later* layer seems to remove it. The right pattern: secrets are passed in when you *run* the container, as environment variables (`docker run -e OPENAI_API_KEY=... myimage`, or a runtime `--env-file` that's itself kept out of git) — the image itself never contains a secret, at any layer, ever. This is the same "config comes from the environment" rule from Doc01's `.env` topic: the code inside the container still calls `os.getenv("OPENAI_API_KEY")` and still fails loudly at startup if it is missing.

Multi-stage builds and image size (good to know, not required for this project): a multi-stage `Dockerfile` uses one stage to build things, and a second, smaller stage that only copies over the final result — keeping the shipped image small by leaving out build tools the running app doesn't need. `.dockerignore` (like `.gitignore`, but for Docker) stops unneeded files from being copied into the image, keeping builds fast and images small. A good starting `.dockerignore`:

```
.git/
.venv/
__pycache__/
.env
*.db
```

Listing `.env` here is a second safety net: even `COPY . .` can't copy your secrets into the image.

`docker-compose` (good to know for running several services locally): once your system needs more than one container (like your API plus a real Postgres database instead of SQLite), `docker-compose.yml` describes how the containers run together — the network between them, shared storage, start-up order — so `docker compose up` starts your whole local setup with one command instead of several separate `docker run`s.

Real-world examples, by situation:

A new teammate joins: instead of a one-page setup guide ("install Python 3.11, then these 14 packages..."), they run `docker build` and `docker run`. Five minutes later the API runs on their laptop exactly like on yours.

Deploying to a cloud platform: most platforms take an image, start containers from it, and set environment variables in their dashboard. Your `Dockerfile` is the whole deploy recipe; no `.env`

file exists on the server.

A missing secret at startup: `docker run contentforge` without `--env-file` should crash at once with `MissingConfigError: OPENAI_API_KEY is not set` (Doc01's startup check). This is the Test Cases row "Container fails to start with a clear error, not a silent hang."

A multi-agent system with a separate MCP tool server: the agent API and the MCP server each get their own image, and `docker compose` starts both on one private network, so the agents can call the tool server by its service name.

Where you'll meet it: the Failure exercise below, Step 3 of Project 5 (containerized and rate-limited), and Doc19, where CI/CD builds the image and releases it. Project 11 and Project 13 are larger multi-agent systems that you can ship the same way. On the road to multi-agent systems, Docker is how "my agents work on my laptop" becomes "my agents run on a server 24 hours a day": the whole Project 4 pipeline goes in one image first, and later, when agents or tools become separate services, each one gets its own container. Real systems: API backends, scheduled jobs, RAG services with their vector store in a second container, and internal tools.

A common mistake: starting uvicorn inside the container with its default host, `uvicorn main:app` (which listens on `127.0.0.1`). What it causes: `127.0.0.1` inside a container means "only this container itself", so `curl localhost:8000` from your computer gets "connection refused" or an empty reply, even though the container logs say the server started. How to spot it: the logs show `Uvicorn running on http://127.0.0.1:8000`. Fix it with `--host 0.0.0.0` in `CMD`, and make sure you passed `-p 8000:8000` to `docker run`.

Quick cheat sheet:

- Image = built recipe result; container = one running copy of it.
- Copy `requirements.txt` and `pip install` *before* `COPY . .`, so code edits don't redo the install.
- Secrets come in at `docker run` (`-e` / `--env-file`), never inside the image. Put `.env` in `.dockerignore`.
- In `CMD`, run uvicorn with `--host 0.0.0.0`, and publish the port with `-p 8000:8000`.
- Data that must survive (a SQLite file) goes on a volume (`-v`).
- Check your image with `docker history <image>` before you call it done.

Caching: what's safe, and what isn't

Caching means saving a past result, so a repeated request doesn't redo expensive work. You store the result under a **key** (a unique label built from everything that affects the result), and next time the same key comes in, you return the saved result instead of calling the model again.

Why this matters for cost specifically: every LLM or embedding call costs money and time. Real traffic repeats itself a lot — the same FAQ question, the same document chunk, the same search. Caching embeddings alone can remove a large amount of wasted spending in a RAG-heavy system,

since the same documents otherwise get re-embedded needlessly. A cached reply also returns in milliseconds instead of seconds. But caching the wrong thing (a reply that depends on live data) trades a small cost saving for silently giving wrong answers, which is a much worse trade.

What's safe to cache: embeddings (the same text with the same model gives the same embedding — nothing to recompute), and replies to fixed, repeated questions where the answer does not change, like "what are your support hours?" with the same system prompt and model. A note on `temperature=0` (temperature is the setting that controls how random the model's word choice is): it makes replies *much more* repeatable, but not perfectly identical every time. Caching such a reply is fine when any one of those replies is an acceptable answer. **What's not safe to cache:** anything that depends on live, changing information — a reply mentioning "today's date," fresh data from a tool call (stock prices, order status), anything personal to one user unless that user is part of the key, or anything where an outdated cached answer would be actively wrong, not just slightly different.

Safe or not, by situation:

Situation	What to do	Why	Small example
Embedding the same document chunks again (re-indexing, restarts)	**Cache**, forever, keyed by model + text	Same input + same model = same vector	Key: `sha256("text-embedding-3-small" + chunk)`
A common FAQ question in a support chatbot	**Cache** with a TTL (time-to-live — how long the entry stays valid)	Answers change rarely, but they do change	FAQ reply cached for 1 hour
A tool result that changes (order status, weather, prices)	**Don't cache**, or use a very short TTL	An old value is a wrong answer	Order status: no cache
A reply that uses one user's private data	Cache only with the user ID **in the key**, or not at all	Otherwise user B can get user A's answer	Key includes `user_id`
Creative writing where variety is the point	**Don't cache**	Users expect a new result each time	"Write me 3 new slogans"
The same long system prompt sent on every call	Use the **provider's prompt caching** (OpenAI applies it automatically to long, repeated prompt beginnings)	You save on input tokens, and the model still writes a fresh answer	Put the fixed system prompt first, the changing user text last

How it works — an embedding cache:

```
import hashlib
from openai import OpenAI

client = OpenAI()
_embedding_cache: dict[str, list[float]] = {}

def _key(model: str, text: str) -> str:
    return hashlib.sha256(f"{model}\n{text}".encode()).hexdigest()

def embed(text: str, model: str = "text-embedding-3-small") -> list[float]:
    key = _key(model, text)
    if key in _embedding_cache:
        return _embedding_cache[key]   # cache hit: no API call, no cost
    response = client.embeddings.create(model=model, input=text)
    vector = response.data[0].embedding
    _embedding_cache[key] = vector     # cache miss: call once, then save
    return vector
```

The key is a hash (a short fixed-length fingerprint) of the model name **and** the text. If you change the model, the old vectors are not reused by mistake. A plain dict lives in memory: it is lost on restart, and it is not shared between two containers. For real use, store the same key → value pairs in a SQLite table (the previous topic), or in **Redis** (a separate, very fast in-memory data server that several containers can share).

Real-world examples, by situation:

A support chatbot FAQ cache with a time limit:

```
import time

TTL_SECONDS = 3600
_reply_cache: dict[str, tuple[float, str]] = {}   # key -> (saved_at, reply)

def cached_reply(key: str) -> str | None:
    item = _reply_cache.get(key)
    if item and time.monotonic() - item[0] < TTL_SECONDS:
        return item[1]
    return None   # missing or too old → call the model
```

Log hits and misses with Doc01's logger (`logger.info("cache hit key=%s", key[:8])`), so you can see whether the cache actually saves money.

RAG over company documents: when HR updates 3 of 500 policy files, only those 3 files' chunks have new text, so only they get new keys and new embedding calls. The other 497 are cache hits.

A delivery app asks "where is my order?": never cache this reply. The order moves every hour; a cached "it's at the warehouse" is a wrong answer to an angry customer.

A multi-agent research pipeline: the researcher agent searches the web for the same topic in several runs during one afternoon. A short TTL (say 15 minutes) on search results saves calls, while the writer agent's final article is never cached, because each request should get its own draft.

Where you'll meet it: Doc08 and Project 3 (embedding many chunks — the biggest, safest win), Project 5 (keeping Project 4's multi-agent pipeline affordable under real traffic), and Doc19, where you watch cost over time and see the cache's effect. In multi-agent systems (Doc11), one user task can mean 5-20 model calls across agents, so a safe cache on the repeated parts (embeddings, fixed lookups, tool results that don't change) often saves more there than anywhere else. Real systems: chatbots with FAQs, RAG over company docs, search APIs, and scheduled jobs that re-process mostly unchanged data.

A common mistake: building the cache key from only the user's question, like `key = question`. What it causes: after you change the system prompt or switch the model, the cache keeps returning old answers made with the old prompt. Worse, if the reply used one user's data, a second user who asks the same question gets the first user's private answer. How to spot it: a prompt change "does nothing" in testing, or users see details that are not theirs. Fix it by putting everything that changes the answer into the key: model, system prompt (or its version), settings, the question, and the user ID when the data is personal.

Quick cheat sheet:

- Cache key = everything that changes the answer: model + prompt/version + settings + input (+ user ID if personal).
- Embeddings: cache freely. Stable FAQ replies: cache with a TTL. Live data and creative output: don't cache.
- `temperature=0` means "more repeatable", not "always identical".
- A dict cache dies on restart and isn't shared; use SQLite or Redis when that matters.
- Log cache hits and misses, so you can prove the cache saves money.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- FastAPI documentation — go through the Tutorial section start to finish.
- Docker — Get Started — the official walkthrough.
- Python `sqlite3` module docs — move to SQLAlchemy once your schema outgrows plain SQL.
- The Twelve-Factor App — read it fully this time, not just the config part from Doc01.

Practice Exercises

Setup for this document's practice code: work inside `12_production_engineering/` (same venv as before — if it's not active, `source .venv/bin/activate`). New packages for this document: `pip install fastapi "uvicorn[standard]" httpx requests` (`httpx` is what FastAPI's `TestClient` runs on).

Where your code lives: all of it under `12_production_engineering/practice/` (`mkdir -p practice`), never loose beside this README. Copy Doc01's `logging_setup.py` in there too, unchanged. Exercises are grouped **by topic, not by difficulty level** — the same convention as Doc01/02/07/09/10/11 — so one topic's growth from basic to intermediate stays visible in one file.

The full file layout, all exercises:

```
practice/
├── chat_route_practice.py          Intermediate
├── chat_schemas.py                   its request/reply models
├── db_failure_and_docker_practice.py  Failure (+ Dockerfile,
│                                     .dockerignore, requirements.txt)
├── health_route_practice.py        Basic
├── health_routes.py                  its APIRouter (Approach 3)
├── logging_setup.py                copied from Doc01, unchanged
├── runs_db.py                        its SQL helper functions
└── sqlite_persistence_practice.py  Real-world + Edge cases
                                     (2 sections)
```

Why each script exists:

- `chat_route_practice.py` — real logic behind a typed route, with 400 vs 500, server-only error logs, and a request ID per request — the exact shape of the Build Task's `POST /run-task`.
- `db_failure_and_docker_practice.py` — breaks the database on purpose to prove the 500 path, then ships the app in a container with secrets only passed in at run time.
- `health_route_practice.py` — the smallest possible FastAPI app, so the app/decorator/uvicorn setup is learned before any real logic sits behind it.
- `sqlite_persistence_practice.py` — saves every run and reads one back by ID (with a 404), then proves bad input never writes a row and a two-write request can't leave half a row.
- `health_routes.py`, `chat_schemas.py`, `runs_db.py` — the small helper files some Intermediate approaches split out, the same split the Build Task's `routes.py`, `schemas.py` and `db/database.py` use.

How to run each exercise: if two exercises below are really about the same thing, save them together in ONE script named after that topic, with each level's version as its own clearly labeled section inside it. Run each topic's file from inside `practice/`, for example: `python`

`health_route_practice.py`, or `uvicorn health_route_practice:app --reload` for the server.

For this document:

- Basic (`health_route`) is its own topic — save it as `practice/health_route_practice.py`.
- Intermediate (`chat_route`) is its own topic — save it as `practice/chat_route_practice.py`.
- Real-world (`sqlite_persistence`) and Edge cases (`validation_422`) both use the same SQLite table and route — build the persistence first, then confirm bad input never writes to it — save them together as `practice/sqlite_persistence_practice.py`, with each level as its own section.
- Failure (`db_failure_and_docker`) is its own topic — save it as `practice/db_failure_and_docker_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to intermediate, side by side.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

Basic — your first FastAPI route

- **What:** one FastAPI route (`GET /health`) with a typed response.
- **Why:** every real service needs a health check, and this is the smallest possible FastAPI app — get it running before anything more complex.
- **When you'll hit this for real:** the literal first route of Project 5.
- **How to code it:** `app = FastAPI()`, `@app.get("/health") def health() -> dict: return {"status": "ok"}`, run with `uvicorn health_route_practice:app --reload`, and hit it with `curl localhost:8000/health`.
- **Save as:** `practice/health_route_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — wrap real logic behind a route

- **What:** a `POST /chat` route wrapping Project 1's chat code, with a Pydantic request model and proper 4xx/5xx replies.
- **Why:** this is the exact pattern — real logic behind a typed route with correct error handling — that Project 5's entire API layer is built from.
- **When you'll hit this for real:** wrapping any of your projects behind a real API, starting with Project 5.

- **How to code it:** a Pydantic `ChatRequest(BaseModel)` with a `message: str` field, a route that calls your Doc04 chat function and returns a `ChatResponse`, with a `try/except` translating internal errors to the right status code.
- **Save as:** `practice/chat_route_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — persist and read back real data

- **What:** a SQLite table storing every run's input/output/timestamp, read back through a `GET /runs` route.
- **Why:** this is the exact save-and-look-up pattern Project 5's run history needs — practice it here on something small first.
- **When you'll hit this for real:** Project 5's Step 2, almost verbatim.
- **How to code it:** a SQLite table with `id, input, output, created_at` columns, an `INSERT` on every `POST /chat` call, and a `GET /runs` route that `SELECT`s and returns them all as JSON.
- **Save as:** `practice/sqlite_persistence_practice.py`, under a `# Real-world` section (this file also holds the Edge cases exercise below, in its own `# Edge cases` section).
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — a request that fails validation

- **What:** a request body that fails Pydantic's checking — confirm FastAPI's automatic 422 reply, and that no partial database write happens.
- **Why:** you need to know, concretely, that bad input never reaches your database — silent partial writes from bad requests are a real, hard-to-notice bug.
- **When you'll hit this for real:** any public-facing route, the moment a client sends malformed JSON.
- **How to code it:** send a request missing a required field with `curl` or `requests`, confirm you get 422 back, then check your database table row count didn't change.
- **Save as:** `practice/sqlite_persistence_practice.py`, under an `# Edge cases` section (this file also holds the Real-world exercise above, in its own `# Real-world` section).
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — a dead connection, and a real container

- **What:** kill the database connection mid-request (or point at a bad path) and confirm a clean 500, not a raw error trace. Then write a working `Dockerfile`, build it, and run the API from the container, with secrets passed in only as environment variables.
- **Why:** both are real production requirements — an API that leaks internal errors to clients is a security problem, and an app that only runs "on your machine" isn't shippable.

- **When you'll hit this for real:** Project 5's own break-scenario tests, almost exactly.
- **How to code it:** point your database connection string at a nonexistent file/path temporarily, hit the route, confirm 500 with a generic message (check your terminal logs for the real detail). Then write the Dockerfile, `docker build`, and `docker run -e OPENAI_API_KEY=... your-image`, confirming it responds identically to the local version.
- **Save as:** `practice/db_failure_and_docker_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Project 4 as an API

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: wrap Project 4 (the multi-agent system) behind a FastAPI service, backed by a database, packaged with Docker.

Requirements:

- FastAPI route(s) exposing Project 4's task-running ability.
- Saved storage (SQLite to start) for run history: input, output, agent log, timestamp, status.
- Structured logging carried over from `01_python_foundations`, now tagged per-request (a request/run ID on every log line).
- A working `Dockerfile`; the app must run correctly with `docker run`, using only environment variables passed in at run time.
- Basic rate limiting on the public route (even a simple in-memory version is fine — write down its limitation).

Inputs: an HTTP POST with a task description (same shape as Project 4's terminal input, now over HTTP).

Outputs: an HTTP reply with the final result; a saved row in the database; tagged log lines.

Constraints: no secret should ever show up in a log line, an error reply, or the Docker image's layers.

Where your code lives: `12_production_engineering/practice/build_task/` — run everything from inside that folder. Project 5 later builds on this same code.

Suggested files:

```
12_production_engineering/practice/build_task/
├── api/
│   ├── main.py        env check, app, request-ID middleware
│   ├── routes.py      /health, /run-task, /runs/{id}, rate limit
│   └── schemas.py     TaskRequest, TaskResponse
```

```
├── db/
│   ├── database.py    connect, insert/finish/fail/fetch a run
│   └── models.py      Run (one saved row's shape)
├── .dockerignore
├── Dockerfile
├── logging_setup.py   copied from Doc01, unchanged
├── requirements.txt
└── test_api.py        one check per Test Cases row
```

Why each file exists:

- `api/main.py` — the one file uvicorn starts; it refuses to start without `OPENAI_API_KEY`, and tags every request with an ID (`chat_route` Approach 4, `db_failure_and_docker` Approach 2).
- `api/routes.py` — the routes on an `APIRouter` (`health_route` Approach 3), with `chat_route`'s 400/500 handling and the rate limiter.
- `api/schemas.py` — the API's request/reply contract in one place, like `chat_schemas.py`.
- `db/database.py` — every SQL statement behind small named functions, like `runs_db.py`, using `validation_422`'s `INSERT` → `lastrowid` → `UPDATE` pieces.
- `db/models.py` — the checked shape of a saved run, like `RunRecord`.
- `.dockerignore`, `Dockerfile` — `db_failure_and_docker`'s, pointed at `api.main:app`.
- `logging_setup.py` — Doc01's `get_logger()`, so logging works the same as in every earlier document.
- `test_api.py` — proves every row of the Test Cases table below in one command, using `TestClient` like `health_route` Approach 2.

Functions/Components to build:

- `routes.py` → `POST /run-task`, `GET /runs/{id}`, `GET /health`
- `db/models.py` → a `Run` model (id, input, output, log, status, created_at)
- a middleware that adds a per-request ID into logs

Expected Behavior

- A valid task request returns a result and can be looked up afterward by ID.
- A bad request body returns 422 with a clear error, no database write.
- An internal failure (fake one) returns 500 with a plain, safe message to the client — full detail only in server-side logs.
- The containerized app behaves the same as the one you ran locally.

Test Cases

Scenario	Expected
Valid POST /run-task	200, result returned, row saved
Bad request body	422, no database write
Fake internal error	500, plain message to the client, full trace in logs only
`docker run` with a required env var missing	Container fails to start with a clear error, not a silent hang
Two quick requests over the rate limit	Second request rejected with 429

Break-It / Debug Preview

- A database connection leak under repeated requests.
- A secret accidentally baked into a Docker image layer (check with `docker history`).
- A timeout that snowballs into a retry storm against your own API.
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- Scaling this to 10x traffic · what you'd cut first under a cost limit · how secrets safely move from environment to container · why "twelve-factor" config matters at this layer specifically.

Move On When

Project 4 is reachable over HTTP, saves state in a real database, and runs from a Dockerfile — not just `python main.py`. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1 or Hint 2**. Ask for the full solution only if you say **"Show me the solution."**